
Ministerul Educației și cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1

Study and Empirical Analysis of Algorithms for
Determining Fibonacci N-th Term

Elaborated:

std. gr. FAF-231

Nicolae MARGA

Verified:

asist. univ.

Cristofor FIȘTIC

Chișinău – 2025

Contents

1	Algorithm Analysis	3
1.1	Objective	3
1.2	Tasks	3
1.3	Theoretical Notes	3
1.4	Introduction	3
2	Implementation	4
2.1	Recursive Method	4
2.2	Dynamic Programming Method	5
2.3	Matrix Power Method	6
2.3.1	Time Complexity Analysis	7
2.3.2	Space Complexity	7
2.4	Binet Formula Method	7
2.5	Fast Doubling Method	9
2.5.1	Fast Doubling Formulas	9
2.5.2	Pseudocode	9
2.6	Memoization with LRU Cache	10
2.7	Generating Function Method	11
2.7.1	Introducing The Generating Function	11
2.7.2	Computing the coefficients	12
2.7.3	Pseudocode	12
2.7.4	Time Complexity Analysis	12
3	Conclusion	13
4	Repository	13

1 Algorithm Analysis

1.1 Objective

Study and analyze different algorithms for determining the Fibonacci n -th term.

1.2 Tasks

- Implement at least 3 algorithms for determining the Fibonacci n -th term.
- Decide properties of input format used for algorithm analysis.
- Define comparison metrics for the algorithms.
- Analyze the algorithms empirically.
- Present the results of the obtained data.
- Deduce conclusions from the analysis.

1.3 Theoretical Notes

The empirical analysis forms the core of this study, where we will execute each algorithm with various inputs and collect performance data. This approach provides better insights into algorithmic behavior.

After collecting data, the results are presented, using graphs to illustrate performance differences. This visualization helps in understanding performance patterns across different input sizes.

1.4 Introduction

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. Formally, the sequence is defined by the recurrence relation:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad \text{for } n \geq 2$$

This sequence appears in various fields of mathematics, computer science, and nature, including algorithms, cryptography, and the growth patterns of plants and animals. The Fibonacci sequence is known for its relation with the Golden Ratio, as the ratio of successive Fibonacci numbers approaches this irrational number as n increases. In this laboratory work, different methods for calculating Fibonacci numbers and analysis of their computational efficiency are covered.

2 Implementation

Six types of algorithms are implemented in Python and analyzed based on execution time, and space complexity.

2.1 Recursive Method

A straightforward approach computing the n -th term recursively:

```
fibonacci(n):  
    if n <= 1:  
        return n  
    return Fibonacci(n-1) + Fibonacci(n-2)
```

The time complexity of this method is exponential $O(2^n)$. This is because each call to `fibonacci(n)` results in two additional calls to `fibonacci(n-1)` and `fibonacci(n-2)`. This leads to an exponential growth in the number of recursive calls, making the recursive method impractical for large values of n .

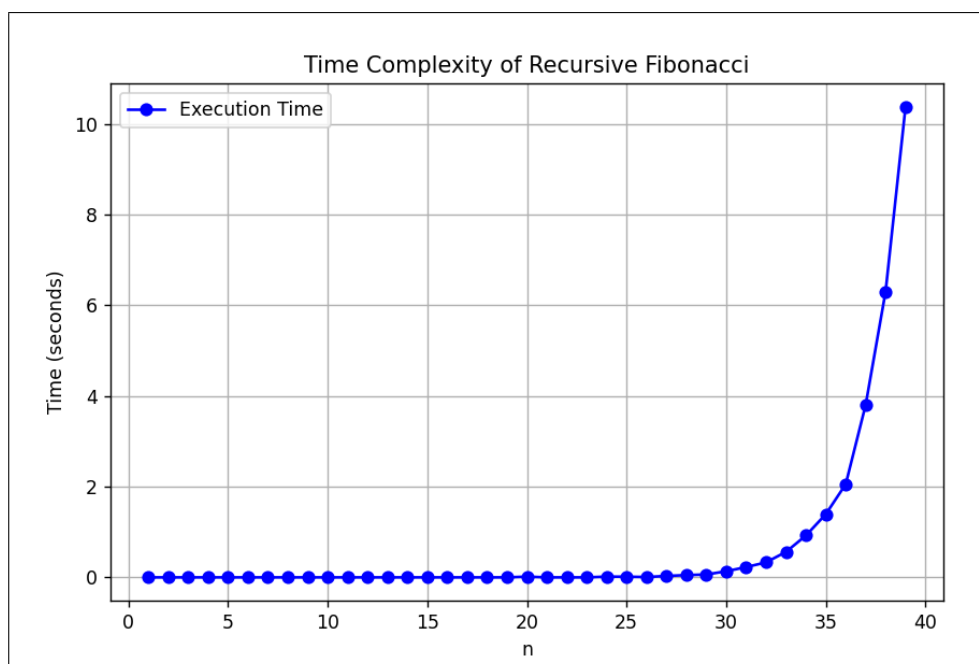


Figure 1: Execution time for Recursive Fibonacci

2.2 Dynamic Programming Method

This method stores previously computed terms to avoid redundant calculations. The algorithm works as follows:

```
Fibonacci(n):  
  A[0] <- 0  
  A[1] <- 1  
  for i <- 2 to n:  
    A[i] <- A[i-1] + A[i-2]  
  return A[n]
```

The time complexity of this algorithm is as follows:

The initialization of the first two values, $A[0]$ and $A[1]$, takes constant time: $O(1)$. The loop runs from $i = 2$ to n , so the number of iterations is $n - 1$. In each iteration, the algorithm performs a constant-time operation, i.e., the addition of two numbers and storing the result in an array element: $O(1)$.

Thus, the total time complexity of the algorithm is $O(n)$, since the loop executes $n - 1$ times and each iteration performs a constant amount of work.

In terms of space complexity:

The algorithm uses an array A of size $n + 1$ to store the Fibonacci numbers, which requires $O(n)$ space.

Thus, the overall time complexity of the Dynamic Programming method is $O(n)$, and the space complexity is also $O(n)$. In Figure 2 the first row contains the n -th Fibonacci term, and the 2-nd row contains the overall execution time to compute the Fibonacci term.

1	500	999	1498	1997	2496	2995	3494	3993	4492	4991	5490	5989	6488	6987	7486	7985	8484	8983	9482	9981
0.0	0.001011	0.0	0.0	0.000993	0.0	0.001007	0.000994	0.001006	0.000999	0.001	0.002	0.002997	0.002002	0.00246	0.003	0.003999	0.003999	0.006001	0.006997	0.007997

Figure 2: Table of Execution time for Dynamic Programming Fibonacci

In Figure 3 it can be seen the graph for execution time of the Dynamic Programming method.

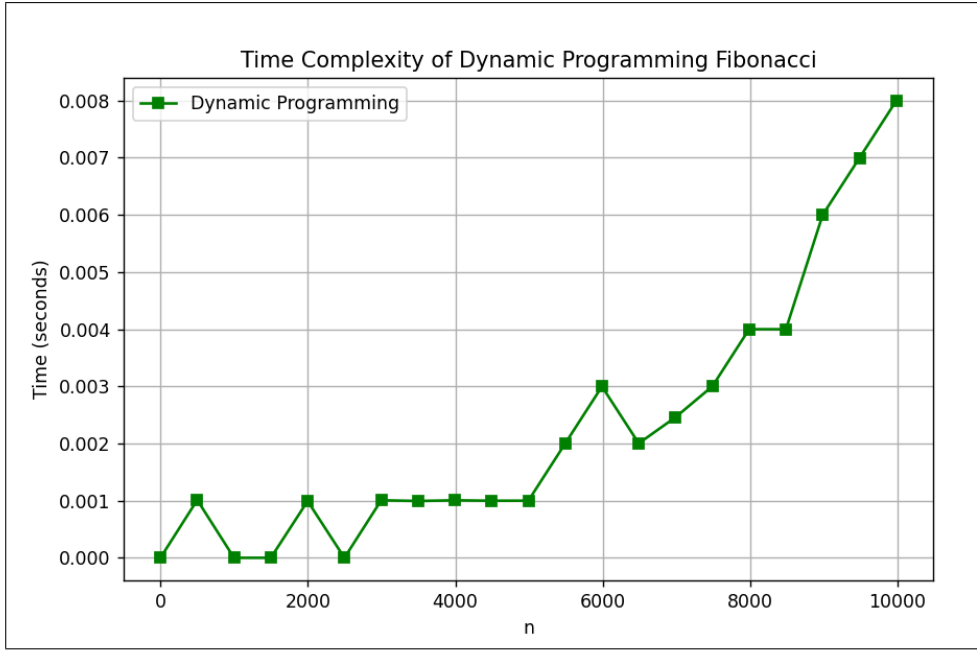


Figure 3: Graph of Execution time for Dynamic Programming Fibonacci

2.3 Matrix Power Method

The Matrix Power Method for calculating Fibonacci numbers uses matrix exponentiation to compute Fibonacci numbers in logarithmic time. The Fibonacci number can be computed using the following matrix relation:

$$\begin{bmatrix} F(n) \\ F(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \cdot \begin{bmatrix} F(0) \\ F(1) \end{bmatrix}$$

This method relies on matrix exponentiation by squaring. The algorithm works as follows:

`Fibonacci(n):`

```
Matrix <- [[0, 1], [1, 1]]
return power(Matrix, n)[0][0]
```

To compute the Fibonacci number at position n , we repeatedly square the matrix $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ and multiply it by itself. This allows the exponentiation to be done in logarithmic steps rather than linear steps.

2.3.1 Time Complexity Analysis

The key in this algorithm is matrix exponentiation by squaring. In this method, the matrix is multiplied by itself in each iteration, and the number of multiplications required to compute the result is reduced by half in each step. This leads to a logarithmic growth in the number of steps required.

The number of iterations required to compute Matrix to the power n is approximately $\log_2 n$. This is because, in each iteration, the exponent is halved.

Thus, the total time complexity of the algorithm is $O(\log n)$, where n is the input to the Fibonacci function.

2.3.2 Space Complexity

The space complexity of the Matrix Power Method is $O(1)$ because we only store the current matrix and a few additional variables to perform the matrix multiplication. The size of the matrix is constant (2×2), and no additional space is used that scales with n .

Therefore, the space complexity is constant.

In summary, the time complexity of the Matrix Power Method for Fibonacci numbers is $O(\log n)$, and the space complexity is $O(1)$.

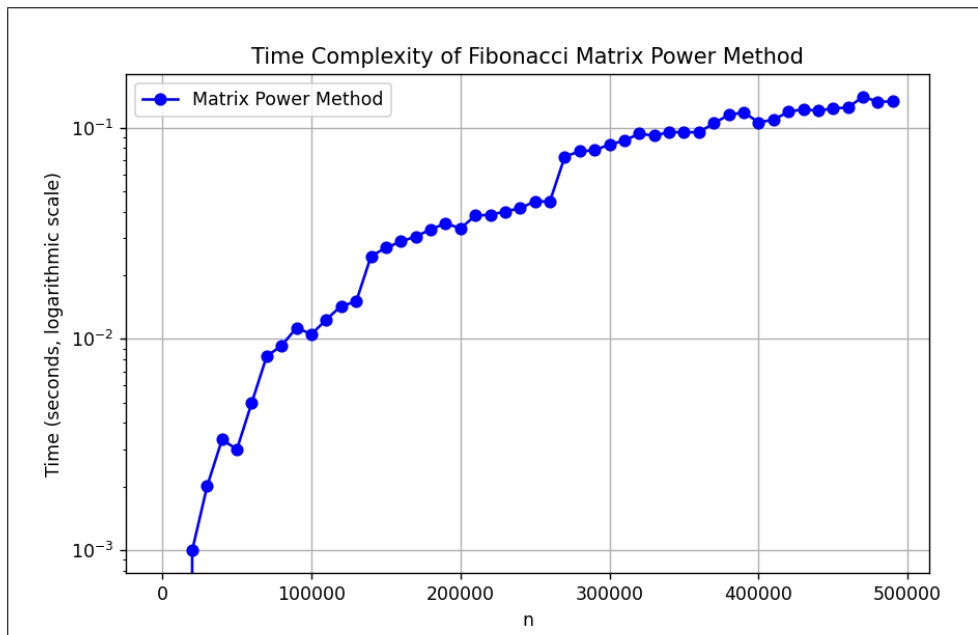


Figure 4: Graph of Execution time for Matrix Power Fibonacci

2.4 Binet Formula Method

The Binet formula provides a closed-form expression for calculating Fibonacci numbers using the Golden Ratio, denoted as φ .

The Fibonacci number F_n for any non-negative integer n can be computed using the following formula:

$$F_n = \frac{\varphi^n - (-\varphi)^{-n}}{\sqrt{5}}$$

However, for large n , the term $(-\varphi)^{-n}$ becomes negligible, so the formula simplifies to:

$$F_n \approx \frac{\varphi^n}{\sqrt{5}}$$

This approximation allows for an efficient calculation of Fibonacci numbers without the need for recursive or iterative computation, especially for large values of n .

The time complexity of the Binet formula method is $O(\log n)$, as it involves only a few mathematical operations, including exponentiation and square root computation.

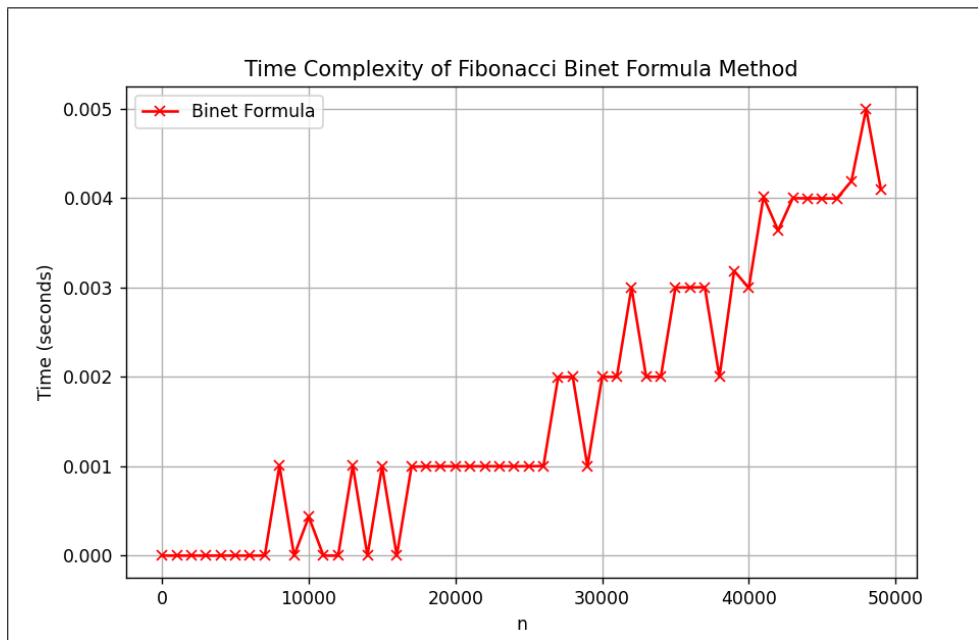


Figure 5: Graph of Time Complexity of Binet's Formula Method

To summarize, the Binet formula is efficient because it has exponentiation φ^n which typically takes logarithmic time to compute in practice depending on the implementation of exponentiation. However, For large n , floating-point precision errors can lead to inaccuracies, and the approximation becomes less accurate due to neglecting small terms. In Figure 6 it is illustrated the loss of accuracy of the Fibonacci terms.

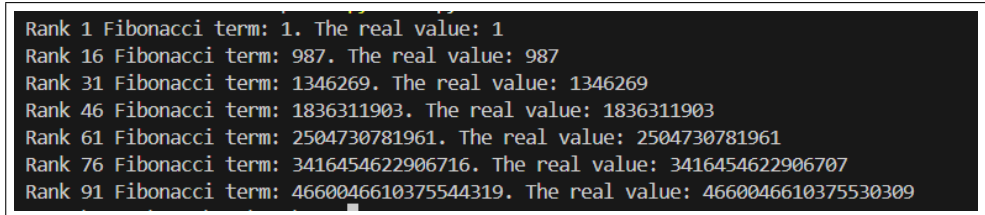


Figure 6: Fibonacci Terms loss of precision

2.5 Fast Doubling Method

The Fast Doubling algorithm reduces most redundant calculations, achieving $O(\log n)$ time complexity.

2.5.1 Fast Doubling Formulas

The algorithm is based on the following identities:

$$F(2k) = F(k) \cdot (2F(k+1) - F(k))$$

$$F(2k+1) = F(k+1)^2 + F(k)^2$$

Using these formulas, we can compute $F(n)$ efficiently using recursive calls.

The Fast Doubling method computes both $F(n)$ and $F(n+1)$ simultaneously.

2.5.2 Pseudocode

```

function fib_fast_doubling(n):
    if n == 0:
        return (0, 1)
    else:
        a, b = fib_fast_doubling(n // 2)
        c = a * (2 * b - a)
        d = b * b + a * a
        if n % 2 == 0:
            return (c, d)
        else:
            return (d, c + d)

```

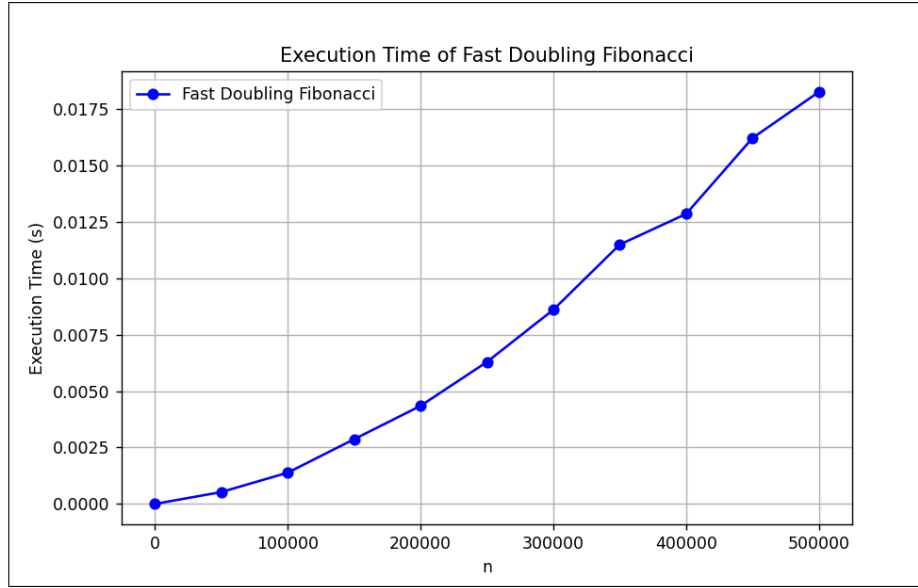


Figure 7: Graph of Time Complexity of Fast Doubling Method

2.6 Memoization with LRU Cache

Memoization is an optimization technique used to speed up recursive functions by storing previously computed results and reusing them when needed. In Python, this can be efficiently implemented using the `@lru_cache` decorator.

The Least Recently Used (LRU) cache stores a fixed number of previously computed values and automatically discards the least recently used ones when the cache is full. For the Fibonacci sequence, a naive recursive implementation has exponential time complexity $O(2^n)$, but by applying memoization, the complexity is reduced to $O(n)$.

In Figure 8 it can be seen the graph of the Time Complexity of the memoization technique.

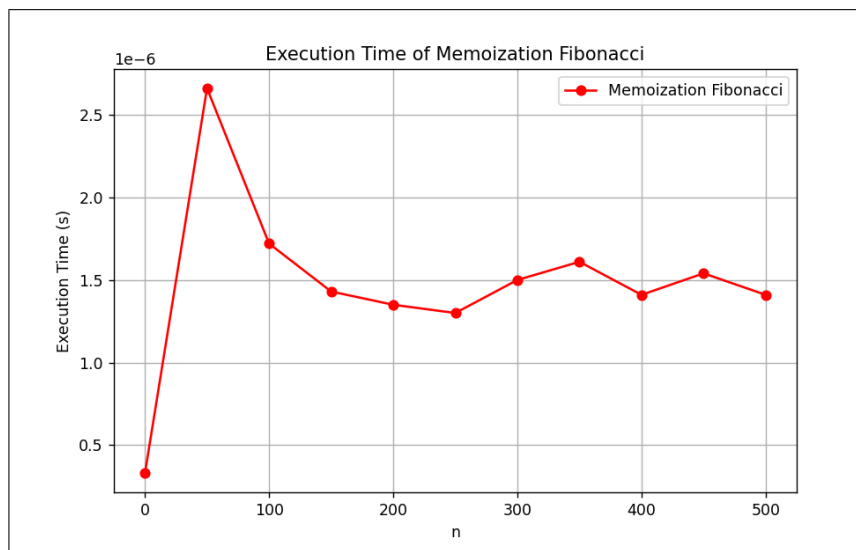


Figure 8: Time Complexity of the Memoization Technique

2.7 Generating Function Method

2.7.1 Introducing The Generating Function

Consider the following power series, centered at $x = 0$:

$$G(x) = \sum_{n=0}^{+\infty} F_n x^n$$

Where F_n is the n -th term of the Fibonacci sequence, and $F_0 = 0$. The domain of convergence of this power series can be found using the ratio test:

$$\lim_{n \rightarrow +\infty} \left| \frac{F_{n+1} x^{n+1}}{F_n x^n} \right| = \phi |x|$$

By the ratio test it $G(x)$ is convergent when $|x| < \frac{1}{\phi}$, thus we can define and consider the domain of G for $x \in (-\frac{1}{\phi}, \frac{1}{\phi})$. In order to reach the closed form of this power series, it can be rewritten as:

$$G(x) = x + \sum_{n=2}^{\infty} (F_{n-2} + F_{n-1}) x^n$$

$$G(x) = x + x^2 \sum_{n=2}^{\infty} F_{n-2} x^{n-2} + x \sum_{n=2}^{\infty} F_{n-1} x^{n-1}$$

$$G(x) = x + x^2 \sum_{n=0}^{\infty} F_n x^n + x \sum_{n=0}^{\infty} F_n x^n$$

$$G(x) = x + x^2 G(x) + x G(x)$$

By rearranging the terms we arrive at the function $G : (-\frac{1}{\phi}, \frac{1}{\phi}) \rightarrow \mathbb{R}$

$$G(x) = \frac{x}{1 - x - x^2}$$

By computing the Maclaurin expansion of G we have:

$$G(x) = \sum_{n=0}^{+\infty} \frac{G^n(0)}{n!} x^n$$

Where G^n is the n -th derivative of G , with $G^0 = G$. Therefore, the coefficients of the Maclaurin expansion of the closed form generating function G must be equal to the coefficients of the Fibonacci sequence.

$$\frac{G^n(0)}{n!} = F_n, \forall n \geq 0$$

2.7.2 Computing the coefficients

2.7.3 Pseudocode

```
x = sp.symbols('x')
f = x / (1 - x - x**2)
coefficient = f.diff(x, n).subs(x, 0) / factorial(n)
```

The code implementation computes the first 20 Taylor coefficients of the generating function

$$f(x) = \frac{x}{1 - x - x^2}$$

using symbolic differentiation in the **sympy** library. For each coefficient, the script calculates the n -th derivative of $f(x)$, evaluates it at $x = 0$, and divides by $n!$ to obtain the Taylor coefficient. The times for computing each coefficient can be seen in the graph of Figure 9.

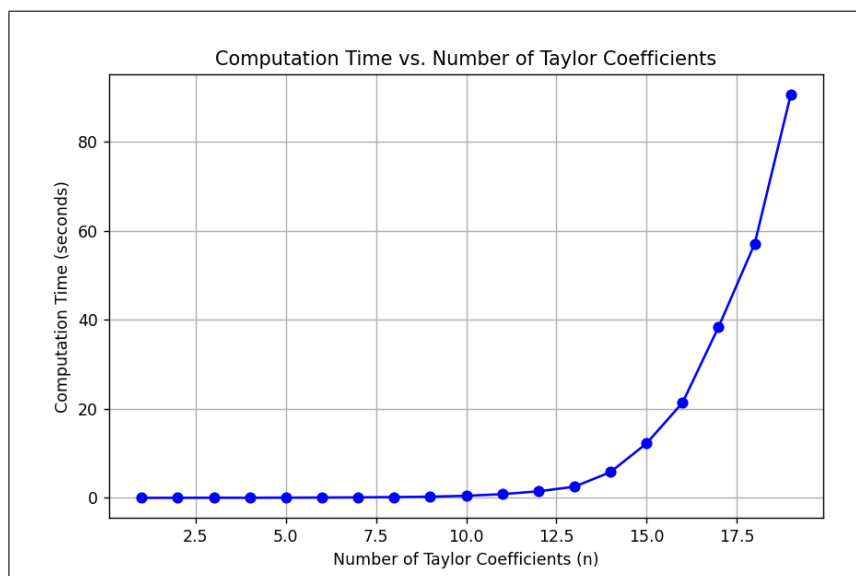


Figure 9: Time Complexity of the Generating Function Technique

2.7.4 Time Complexity Analysis

Computation of the factorial is done using the `sympy.factorial` method, which consists of two algorithms. For small arguments a precomputed look up table is used, and for a larger input the Prime-Swing method is used. It has an overall Time-Complexity of $O(n \log n)$.

However, computing the n -th derivative symbolically requires a lot of operations as n grows, this is because at each step of the differentiation the number of terms increases for rational functions which includes also this generating function. Faster method for computing the n -th derivative will be using numerical methods such as the finite-difference method, but the trade-off includes the loss of accuracy of the result.

3 Conclusion

The empirical analysis of the seven Fibonacci calculation algorithms showcase mainly a performance hierarchy. The recursive method performs poorly with $O(2^n)$ complexity, making it impractical for large inputs, and its use mostly is to showcase how recursion works. Dynamic programming achieves $O(n)$ time complexity by eliminating redundant calculations.

Matrix power and fast doubling methods demonstrate better $O(\log n)$ performance. Binet's formula has the issue of precision errors for large inputs despite its logarithmic complexity. Memoization technique manages to transform exponential recursion into a linear-time algorithm. The Generating Function method also is rather impractical for large inputs, as it requires computing the n -th derivative of a rational function.

This laboratory work illustrates how algorithmic approach selection significantly impacts performance when computing the same mathematical sequence, with logarithmic methods providing the best balance of efficiency and accuracy for large Fibonacci calculations. Dynamic Programming, Matrix Multiplication and Fast Doubling are the most practical for large inputs.

4 Repository

The source code for this laboratory work can be found here:

https://github.com/xnikug/AA_Labs/tree/lab1